Docker Cheat Sheet

Commandes, Dockerfile, Compose, Swarm, monitoring, sécurité

01 BASIC COMMANDS

`docker run <image>`	= docker create <image> + docker start <conteneur>
`docker run -d <image>`	démarrer en arrière-plan
`docker ps`	lister les conteneurs actifs (affiche aussi les id)
`docker ps -a`	lister en plus les conteneurs inactifs et arrêtés
`docker attach <conteneur>`	s'attacher à nouveau à un conteneur (id ou nom)
`docker run --name <name> httpd`	personnaliser le nom
`docker rename <conteneur> <name>`	renommer un conteneur
`docker logs <conteneur>`	afficher les logs en mode détaché (-f : follow, -n n : n dernières lignes)
`--rm`	supprime automatiquement le conteneur à sa sortie (option de docker run)

02 CONTAINERS

`docker stop <conteneur>`	stopper un conteneur
`docker restart <conteneur>`	redémarrer
`docker rm <conteneur>`	supprimer un conteneur inactif
`docker rm -f <conteneur>`	stopper et supprimer un conteneur actif
`docker container prune`	nettoyer tous les conteneurs inactifs

03 IMAGES

`docker images`	lister les images locales
`docker pull <image>`	récupérer une image du Hub. Ex : pull node:18 met à jour node mais pas ses conteneurs !
`docker rmi <image>`	supprimer localement une image
`docker image prune`	supprimer les images sans tag (untagged)
`docker image prune -a`	supprimer toutes les images non utilisées

04 PORT MAPPING

`docker run -p <port_hôte>:<port_conteneur> <image>`	publier un port du conteneur sur la machine hôte

- 80 est le port par défaut d'un conteneur HTTP.
- Accès via localhost:<port_machine_hôte>.

05 EXÉCUTION DANS UN CONTENEUR

`docker exec <conteneur> <commande>`	exécuter une commande à l'intérieur du conteneur (communiquer, debugger pendant que l'app tourne)
`docker exec -it <conteneur> bash`	entrer dans un conteneur en lançant un terminal bash

```
docker exec apache cat /usr/local/apache2/htdocs/index.html
```

- -i et -t : lient l'entrée standard du processus bash au terminal de la machine hôte.
- Ctrl + D pour sortir du conteneur.
- Images distroless : images ne contenant ni shell, ni arborescence, ni gestionnaire de paquets, etc.

06 ÉTAPES POUR CRÉER UNE IMAGE

1	Démarrer d'une image de base
2	Copier les sources de l'application
3	Installer les dépendances
4	Indiquer la commande principale

07 DOCKERFILE

`FROM`	image parente
`RUN`	exécuter des commandes shell
`COPY <src> <dest-image>`	copier de nouveaux fichiers dans l'image
`ADD <src> <dest>`	télécharge src si c'est une URL, sinon décompresse (format tar)
`CMD ["commande", "paramètre", ...]`	commande par défaut (remplaçable)
`ENTRYPOINT ["commande", "paramètre", ...]`	commande nécessaire au démarrage du conteneur, ne peut pas être remplacée
`WORKDIR <chemin>`	change le répertoire de travail de l'image (≈ cd)
`EXPOSE <port> ...`	indique les ports réseau exposés par le conteneur
`ENV <clé>=<valeur>`	crée une variable d'environnement, remplaçable avec docker run -e
`LABEL <clé>=<valeur>`	métadonnées (data about data)
`docker run <image> <commande>`	remplacer la commande par défaut (CMD)
`docker build -t <image> <build-context>`	image = nom[:tag] ; contexte = dossier où se trouve le Dockerfile

08 LAYERS

`docker history <image>`	afficher l'historique d'une image (chaque ligne = une layer)

- Une image est désignée par un pointeur vers son dernier layer.
- Chaque layer contient le différentiel par rapport à la layer précédente.
- Docker utilise un système de fichiers Union pour fusionner les layers et former l'image finale → optimisation du stockage et layers réutilisables.
- À l'instanciation d'un conteneur, une layer Read/Write est créée : elle contient les modifications propres au conteneur (infos volatiles). Cela permet de lancer plusieurs conteneurs à partir de la même image.

09 COMMANDES DE LAYER

`docker diff`	affiche le différentiel du conteneur par rapport à l'image
`docker commit`	sauvegarde l'état du conteneur dans une image
`docker save`	sauvegarde images et layers dans un TAR
`docker load`	charge depuis un TAR

`docker export`	exporte le système de fichiers d'un conteneur dans un TAR
`docker import`	crée une image depuis le système de fichiers d'un TAR, sans historique (l'image reflète l'état du conteneur à l'export)

- Dive : outil third-party qui permet de naviguer dans les layers.

10 BONNES PRATIQUES

- Nettoyer les dépendances inutiles.
- Utiliser plusieurs instructions par commande pour éviter la création de multiples layers.
- Utiliser un .dockerignore : les fichiers inutiles à l'exécution ne sont pas inclus dans l'image finale.
- Utiliser ENTRYPOINT ou CMD.
- COPY est préférable à ADD.
- Utiliser plusieurs conteneurs (un rôle par conteneur).

11 MULTI-STAGE BUILD

On build un projet dans une image X, puis on sert les fichiers statiques produits dans une image Y finale.

```
# 1 – étape de build
FROM X:tag AS builder
WORKDIR /app
...

# 2 – image finale
FROM Y:tag
COPY --from=builder src-path dest-path
```

12 DOCKER HUB

`docker login`	se connecter depuis le terminal
`docker tag <image> username/repo:latest`	tagger l'image
`docker push username/repo:latest`	pousser l'image

13 LES VOLUMES

- Accessible depuis le conteneur ; peut être monté dans le système de fichiers des conteneurs à leur création (runtime).
- Son contenu est accessible à l'emplacement choisi depuis le conteneur.
- On peut créer des volumes anonymes.
- Il existe différents drivers (types) de volumes, ex : SSHFS (stockage sur un serveur accessible par SSH).

`docker volume create <nom>`	créer
`docker volume ls`	lister
`docker run -v <nom>:<chemin> <image>`	monter en lecture/écriture
`docker run -v <nom>:<chemin>:ro`	monter en lecture seule
`docker volume rm <nom>`	supprimer un volume inutilisé
`docker volume prune`	nettoyer les volumes anonymes inutilisés
`docker volume prune -a`	nettoyer tous les volumes inutilisés

14 LES BIND MOUNTS

- Lier les fichiers de sa machine hôte à l'environnement d'exécution du conteneur.

`docker run -v "<chemin-hôte>:<chemin-conteneur>" <image>`	créer un bind mount

- Utile aussi pour injecter un fichier de configuration dans un conteneur.

15 LES RÉSEAUX

- On peut assigner des réseaux à des conteneurs. Réseau par défaut : bridge.

`docker network ls`	lister les réseaux
`docker network create <nom>`	créer un réseau
`docker run --network <nom> <image>`	assigner un conteneur à un réseau dès sa création
`docker network connect <nom> <conteneur>`	assigner à un conteneur déjà démarré
`docker network disconnect <nom> <conteneur>`	déconnecter un conteneur du réseau
`docker inspect <nom>`	détails sur tout objet Docker : pour un réseau, l'IP du sous-réseau ; pour un conteneur, la liste des réseaux assignés et son IP
`docker network rm <nom>`	supprimer un réseau (s'il n'a plus de conteneur assigné)
`docker network prune`	nettoyer tous les réseaux inutilisés

16 LES ALIAS & TYPES DE RÉSEAUX

`docker run --network <nom> --network-alias <alias> <image>`	créer un alias réseau
`docker network connect --alias <alias> <nom> <conteneur>`	ajouter un alias à la connexion

- Le nom (--name) d'un conteneur sert aussi d'alias.

`bridge`	réseau par défaut
`host`	désactive l'isolation et remonte les conteneurs sur l'hôte
`overlay`	connecte des conteneurs d'hôtes différents (cluster swarm)
`none`	désactive la stack réseau du conteneur

17 DOCKER COMPOSE

- Utilise un fichier de configuration compose.yaml (ou compose.yml), présent dans le répertoire courant lors d'une commande docker compose.
- Il décrit un projet, sa configuration et son déploiement. Sensible à l'indentation.
- Si le fichier est modifié, un simple docker compose up suffit (pas besoin de stopper/supprimer/redémarrer les conteneurs).
- Les conteneurs sont appelés services.

```
services:
  container-nginx:
    image: nginx
  container-apache:
    image: httpd
```

`docker compose up (-d)`	démarrer l'application = create + start
`docker compose ps`	lister les conteneurs du projet (préfixés par le nom du dossier)
`docker compose stop / restart / rm / down`	rm : supprime les inactifs ; down : stop + suppression

`docker compose exec <service> <commande>`	attache par défaut l'entrée standard au terminal, pas besoin de -it
`docker compose build / up --build`	forcer le build et mettre à jour l'image (non reconstruite si elle existe déjà)

Ports :

```
ports:
  - "8088:80"
```

Build depuis un Dockerfile (pas besoin de la clé image) :

```
services:
  container-nginx:
    build: <chemin-docker-file>
```

Volumes (les volumes ne sont pas supprimés avec down, il faut forcer avec -v) :

```
services:
  ...
  volumes:
    - ubuntu-data:/home

volumes:
  ubuntu-data:
```

Bind mounts :

```
volumes:
  - <chemin-hôte>:<chemin-container>
```

Réseaux : Compose assigne un réseau par défaut aux services, appelé "<projet>_default".

```
services:
  ...
  networks:
    net1:
      aliases:
        - foo

networks:
  net1:
    driver: bridge
```

Dépendances entre services : l'ordre de déclaration n'a pas d'importance, Docker les démarre en parallèle. Pour imposer un ordre : clé depends_on.

```
services:
  ...
  depends_on:
    - mydatabase
```

Variables d'environnement : directement, ou via un fichier .env avec la clé env_file.

```
services:
  ...
  environment:
    - API=abc
```

```
services:
  ...
  env_file: .env
```

18 DOCKER SWARM

▸ Alternative à Kubernetes ; scalabilité horizontale.

- Conteneurs distribués sur plusieurs nœuds → pas de SPOF.
- Types de nœuds : manager (gère et répartit les tâches) / worker.
- En production : plusieurs serveurs et plusieurs managers. En développement : un seul nœud manager (lui-même worker).
- Algorithme Raft : N/2 + 1 managers (le quorum) doivent fonctionner et être d'accord pour exécuter une nouvelle tâche → utiliser un nombre impair de managers.

`docker swarm init`	créer le 1er nœud du cluster (1er manager)
`docker info`	vérifier l'activation de swarm
`docker swarm join`	ajouter un nœud (commande utilisée sur l'autre machine)
`docker swarm join-token manager`	obtenir le token d'accès du nœud 1 pour ajouter un manager ; renvoie la ligne à taper sur la machine 2
`docker swarm join-token worker`	idem pour un worker
`docker node ls`	lister les nœuds d'un cluster
`docker swarm leave`	le nœud quitte le cluster
`docker node rm <node>`	un manager éjecte un nœud

19 LES SERVICES

- Démarrer la scalabilité horizontale. En cas de crash d'un conteneur ou d'un nœud, les conteneurs sont recréés automatiquement sur d'autres nœuds.

`docker service create --replicas <N> --name <nom> <image>`	conteneurs répartis automatiquement sur les nœuds disponibles
`docker service ls`	lister les services du cluster swarm
`docker service ps <nom>`	lister les conteneurs d'un service
`docker service rm <nom>`	supprimer un service et ses conteneurs
`docker service update --image <image> <nom>`	mettre à jour l'image d'un service, conteneur par conteneur pour éviter l'interruption
`docker service rollback <nom>`	annuler la mise à jour la plus récente
`docker service scale <nom>=<N>`	changer le nombre de conteneurs

20 RÉSEAU ET LOAD BALANCING

- Les conteneurs sont virtuellement sur le même réseau.
- Le réseau par défaut, de type overlay, s'appelle ingress.

`docker network create -d overlay <nom>`	créer un réseau overlay
`docker service create --network <nom> <image>`	assigner le service au réseau

Load balancing : sur un réseau de type overlay, les requêtes envoyées à un port publié sont automatiquement transférées à un conteneur disponible, même s'il est situé sur un autre nœud.

21 LES STACKS

- Déployer plusieurs services en même temps sur un cluster swarm, à partir d'un compose.yaml.
- On ajoute la clé deploy → replicas : N.

```
deploy:
  replicas: N
```

`docker stack deploy -c compose.yaml <nom>`	démarrer les services d'une stack dans swarm

`docker stack ls`	lister les stacks
`docker stack services <nom>`	lister les services d'une stack
`docker stack ps <nom>`	lister les conteneurs d'une stack
`docker stack rm <nom>`	supprimer une stack et arrêter les services

- Alternatives : Kubernetes, OpenShift, Nomad.
- Pour aller plus loin : gestion des nœuds (changer le type, mettre en pause), limiter les ressources, docker secrets...

22 MONITORING

- Logs : servent à monitorer ou diagnostiquer un problème (stdout et stderr). Ajouter -f pour suivre en temps réel (similaire à tail -f).

`docker logs <conteneur>`	logs d'un conteneur
`docker compose logs`	logs des services compose
`docker service logs <service>`	logs d'un service swarm
`docker stats`	consulter les ressources utilisées par le conteneur (CPU, RAM...)

Healthcheck : vérifier l'état de santé en exécutant régulièrement une commande (dans le Dockerfile). Code de sortie ≠ 0 → unhealthy (dysfonctionnel).

```
HEALTHCHECK CMD curl -f http://localhost/ || exit 1
```

- -f : curl échoue si le code HTTP est ≥ 400 ; curl retourne 0 si réussi ; || exit 1 : en cas d'échec, exit 1 est exécuté.
- docker ps ajoute alors un statut healthy / unhealthy / starting au conteneur.
- Limiter les ressources avec --cpus, --memory... (possible aussi depuis la configuration docker compose).
- Il existe des outils annexes de monitoring.

23 REDÉMARRAGE

- Docker ne redémarre pas automatiquement les conteneurs stoppés. Pour limiter les interruptions de service, on adopte une politique de redémarrage :
- always
- unless-stopped : ne redémarre pas si explicitement stoppé par l'utilisateur
- on-failure : si le code de sortie ≠ 0

`docker run -d --restart <politique> <image>`	avec docker run
`restart: <politique>`	dans compose.yaml

24 SÉCURITÉ

- Plusieurs niveaux de sécurité : hôte, démon, images, conteneurs.
- Hôte : anti-virus, accès...
- Démon : endpoint, rootless mode.
- Images : dépendances (docker scout pour détecter les vulnérabilités), images malveillantes, digest : image:tag@digest (code généré à partir du contenu exact de l'image).
- Conteneurs : ports publiés, réseaux, volumes.
- docker bench : permet de vérifier la sécurité d'une installation Docker.

25 BACKUP

- Il faut sauvegarder : la config de déploiement, les images, les données des conteneurs.
- Config de déploiement : sauvegarder les scripts pour démarrer tous les conteneurs avec docker compose.
- Images : stockées dans un registre Docker externe, dissocié des serveurs de production.

- Conteneur : sauvegarder l'état avec docker commit ; docker exec pour extraire directement les données à sauvegarder.
- Volumes et bind mounts : utiliser un cron (automatise l'exécution de tâches à intervalles de temps) pour sauvegarder les fichiers des volumes.

26 NETTOYER DOCKER

`docker system prune`	supprime tous les conteneurs arrêtés, réseaux inutilisés, images...
`docker system prune -a`	idem, + toutes les images non utilisées par au moins un conteneur actif